

Evaluation von Prompt-Engineering-Techniken für LLM-as-a-Judge: Second-Level Judging und Dynamic Prompting

Semesterprojekt-Proposal

Esmo Morina, Tatiana Malkina

29. Juni 2026

1 Introduction

1.1 Bereich der Arbeit

Mit dem Einsatz großer Sprachmodelle (LLMs) in produktiven Systemen wird die zuverlässige Bewertung ihrer Ausgaben zu einer zentralen Herausforderung [1], [2]. Fehlbewertungen können sich nicht nur auf Kosten und Ressourceneinsatz auswirken, sondern auch die Qualität realer Anwendungen beeinträchtigen, beispielsweise durch falsche oder irreführende Antworten [3]. Trotz der Vielzahl bestehender Ansätze bleibt die Entwicklung von Evaluationsverfahren, die gleichzeitig skalierbar, reproduzierbar und robust gegenüber variierenden Bedingungen sind, eine wichtige Aufgabe im Bereich der LLM-Evaluation [4]. Vor diesem Hintergrund hat sich das Paradigma des LLM-as-a-Judge etabliert, bei dem Sprachmodelle selbst zur automatisierten Bewertung von Modellantworten eingesetzt werden [1]. Während dieser Ansatz eine effiziente Skalierung von Evaluationsprozessen ermöglicht, wirft er zugleich Fragen hinsichtlich der Zuverlässigkeit der erzeugten Bewertungen auf. Aktuelle Studien zeigen, dass die Bewertungsergebnisse von Faktoren wie Prompt-Formulierung, Modellwahl und Aufgabenstruktur beeinflusst werden können, was zu einer erhöhten Variabilität der Resultate führen kann [3], [5], [6]. Ein Ansatz zur Verbesserung der Bewertungsqualität ist das sogenannte Second-Level Judging, bei dem eine initiale Bewertung durch einen weiteren Prüfungsschritt verifiziert wird. Dabei wird die ursprüngliche Bewertung einer zusätzlichen Analyse unterzogen, um deren Konsistenz und Nachvollziehbarkeit zu überprüfen. Die Idee einer nachgelagerten Prüfung wurde unter anderem von Russinovich et al. aufgegriffen und später auch im Kontext von LLM-as-a-Judge untersucht [3], [7]. Parallel dazu nimmt das Prompt Engineering eine Schlüsselrolle ein, da es das Verhalten von Sprachmodellen maßgeblich determiniert [1]. Insbesondere adaptive Ansätze wie das Dynamic Prompting konnten in diversen Anwendungsszenarien bereits signifikante Steigerungen der Leistungsfähigkeit erzielen [8].

...

1.2 Motivation

Large Language Models (LLMs) haben sich zu einer zentralen Technologie entwickelt. Dadurch steigt der Bedarf an skalierbaren Evaluationsverfahren, da menschliche Evaluationen zwar qualitativ hochwertig sind, jedoch mit erheblichem Zeit- und Kostenaufwand verbunden sind [1].

Ein bedeutender Forschungsbereich ist daher **LLM-as-a-Judge**, bei dem Sprachmodelle automatisch die Antworten anderer Modelle bewerten oder Systeme miteinander vergleichen. Solche Ansätze werden zunehmend für Benchmarking, Qualitätskontrolle und automatisierte Modellvergleiche genutzt. Die Qualität dieser Bewertungen hängt jedoch nicht nur vom eingesetzten Modell, sondern auch stark vom Prompt-Design ab [3], [5].

Ein aktueller Ansatz ist das **Second-Level Judging**, bei dem ein zweites Judge-Modell die Entscheidung und Begründung eines ersten Judges überprüft, um Fehler zu erkennen und die Zuverlässigkeit zu erhöhen. Erste Arbeiten berichten Verbesserungen durch diesen zweistufigen Prozess, insbesondere bei der Reduktion fehlerhafter Urteile [7]. Neuere Studien zeigen jedoch widersprüchliche Ergebnisse und kommen zu dem Schluss, dass Second-Level Judges die Gesamtleistung in vielen Fällen sogar verschlechtern können [3].

Parallel dazu zeigen Studien in anderen Bereichen, etwa bei der automatisierten Codefehlererkennung, dass **Dynamic Prompting** die Leistung von Sprachmodellen deutlich verbessern kann [8]. Ob sich dieser Ansatz auf LLM-as-a-Judge-Systeme übertragen lässt, wurde bislang kaum untersucht.

Zusätzlich zu allgemeinen Evaluationsrahmen und Benchmarks für LLM-basierte Bewertungsmodelle [2] konzentriert sich diese Arbeit speziell auf den Einfluss von Prompting-Strategien innerhalb solcher Bewertungs-Setups.

Ziel dieser Arbeit ist es, Prompting-Techniken für **LLM-as-a-Judge-Systeme** systematisch zu vergleichen. Im Fokus stehen dabei **Second-Level Judging** und **Dynamic Prompting**.

...

2 Verwandte Forschung

Die Nutzung großer Sprachmodelle als Evaluatoren (LLM-as-a-Judge) wird zunehmend als skalierbare Alternative zu klassischen Bewertungsmethoden betrachtet [1], [5]. Studien belegen, dass leistungsstarke Modelle in der Lage sind, die Ausgaben anderer Sprachmodelle automatisiert zu validieren und dabei eine hohe Korrelation mit menschlichen Urteilen erreichen [5]. Darauf aufbauend wurden Verfahren entwickelt, die mittels strukturierter Prompt-Strategien eine präzisere Ausrichtung an menschlichen Bewertungskriterien anstreben [9]. In der Literatur wird dieser Ansatz häufig als parametrisierte Bewertungsfunktion formalisiert **gu2026survey**:

$$\text{Score} = f(\text{model}, \text{prompt}, \text{task}, \text{context}) \quad (1)$$

Die Bewertungsqualität hängt dabei maßgeblich vom gewählten Modell, dem Prompt, der spezifischen Aufgabe sowie dem Kontext ab und reagiert entsprechend sensitiv auf Variationen dieser Faktoren [1], [6]. Parallel dazu werden Verfahren untersucht, die die Leistungsfähigkeit von Sprachmodellen durch zusätzliche Analyse- oder Verfeinerungsschritte verbessern sollen. Beispiele hierfür sind Reflexion [10], Self-Refine [11] sowie Dynamic Prompting [8]. Ein prominentes Beispiel ist das Dynamic Prompting, bei dem zunächst eine Zwischenanalyse erzeugt und anschließend als zusätzlicher Kontext in den Prompt integriert wird [8]. Während empirische Ergebnisse zeigen, dass dieser Ansatz die Qualität generierter Antworten verbessern kann, wurde sein Einfluss auf Aufgaben der automatisierten Evaluation (LLM-as-a-Judge) bislang nur unzureichend untersucht [8].

Daneben existieren mehrstufige Bewertungsverfahren wie Second-Level Judging, bei dem eine Primärbewertung erneut analysiert und gegebenenfalls korrigiert wird. Ziel ist die Steigerung der Reliabilität des Urteils. Empirische Studien belegen jedoch, dass solche Verfahren keine konsistente Verbesserung garantieren und in spezifischen Modell-Prompt-Konfigurationen sogar kontraproduktive Effekte erzielen können [3].

Neuere Arbeiten erweitern diese Idee zum Meta-Judging, bei dem Modelle nicht nur Antworten, sondern auch bereits erzeugte Evaluationen prüfen und somit als „Richter der Richter“ fungieren. Diese Ansätze zielen darauf ab, die Konsistenz automatisierter Bewertungssysteme zu erhöhen, und zeigen in isolierten Konfigurationen bereits Leistungsgewinne. Bestehende Studien konzentrieren sich jedoch häufig auf komplexe Architekturen wie Multi-Agenten-Systeme oder präferenzbasiertes Training, während praktisch leichter umsetzbare Varianten wie das Second-Level Judging bislang unzureichend erforscht sind [4].

Insgesamt zeigt sich, dass zusätzliche Analyseebenen – sowohl innerhalb des Reasoning-Prozesses als auch durch separate Prüfungsschritte – die Entscheidungsqualität maßgeblich beeinflussen können, deren exakte Wirkweise jedoch noch nicht eindeutig geklärt ist. Insbesondere bleibt offen, unter welchen Bedingungen Second-Level-Ansätze die Verlässlichkeit von LLM-basierten Evaluatoren tatsächlich optimieren. Dies macht eine systematische Untersuchung verschiedener Implementierungen – insbesondere dynamischer Prompting-Ansätze und mehrstufiger Verfahren – zu einer dringlichen Forschungsaufgabe.

...

3 Problemstellung

3.1 Beitrag der Arbeit

Im Rahmen dieser Arbeit wird die Zuverlässigkeit automatisierter Bewertungsverfahren auf Basis des Ansatzes *LLM-as-a-Judge* untersucht, wobei der Fokus insbesondere auf mehrstufigen Strategien liegt. Bestehende Studien liefern hierzu widersprüchliche Ergebnisse: Während mehrstufige Verfahren wie *Dynamic Prompting* die Qualität bei Analyseaufgaben (z. B. Codeanalyse und Schwachstellenerkennung) nachweislich steigern können [8], führen analoge Ansätze im Kontext der Evaluation nicht zwangsläufig zu den erwarteten Verbesserungen [3].

Insbesondere zeigt der Ansatz des *Second-Level Judging*, der eine zusätzliche Überprüfung initialer Bewertungen vorsieht, trotz seiner konzeptionellen Einfachheit häufig instabile oder sogar schlechtere Ergebnisse [3]. Im Gegensatz dazu erzielen komplexere Methoden wie das *Meta-Judging* oftmals eine höhere Übereinstimmung in der Bewertung [4]. Dieses Spannungsfeld stellt ein in der Literatur bislang unzureichend geklärtes Problem dar.

Ziel dieser Arbeit ist die systematische Untersuchung dieser Diskrepanzen sowie die Identifikation von Bedingungen, unter denen mehrstufige Ansätze die Zuverlässigkeit der Bewertung maßgeblich beeinflussen.

Zur Umsetzung wird eine experimentelle Pipeline für binäre Klassifikationsaufgaben entwickelt, die den systematischen Vergleich verschiedener Konfigurationen des Bewertungsprozesses ermöglicht. Dabei werden insbesondere der Einfluss von *Second-Level Judging* sowie verschiedener Prompting-Strategien – einschließlich *Dynamic Prompting* – auf die Stabilität und Konsistenz der Bewertungen analysiert. Zusätzlich erfolgt ein Vergleich von Modellen unterschiedlicher Leistungsfähigkeit, um die Korrelation zwischen Bewertungsqualität und Rechenaufwand zu untersuchen.

Als Datengrundlage dienen etablierte Benchmarks, ergänzt durch einen dedizierten Satz eigener Beispiele. Zur Evaluation werden klassische Metriken (Precision, Recall, F1-Score), das Maß der Übereinstimmung (Cohen's Kappa) sowie Stabilitätskennzahlen auf Basis der Variabilität über mehrere Iterationen eingesetzt.

Die Ergebnisse der Arbeit sollen dazu beitragen, die Kausalmechanismen hinter den beobachteten Diskrepanzen zu isolieren und evidenzbasierte Empfehlungen für den Einsatz mehrstufiger Bewertungsansätze abzuleiten.

...

3.2 Forschungsfragen

Zur Erreichung dieses Ziels ergibt sich die folgende **Hauptfrage**:

Wie beeinflussen mehrstufige Bewertungsansätze wie *Second-Level Judging* und *Dynamic Prompting* die Qualität, Stabilität und Effizienz von LLM-basierten Bewertungssystemen im Vergleich zu einem einstufigen Baseline-Ansatz, insbesondere bei Antworten, die durch Prompt-Injection- oder Jailbreak-Eingaben erzeugt wurden?

Teilfragen

1. Wie unterscheiden sich *Second-Level Judging* und *Dynamic Prompting* im Vergleich zu einem einstufigen Baseline-Ansatz hinsichtlich der Stabilität und Konsistenz der Bewertungsergebnisse?
2. In welchem Ausmaß verbessern *Second-Level Judging* und *Dynamic Prompting* die Bewertungsqualität bei Antworten, die durch Prompt-Injection- oder Jailbreak-Eingaben erzeugt wurden?
3. Welcher der untersuchten Ansätze erzielt die höchste Übereinstimmung mit den Ground-Truth-Labels sicherheitsrelevanter Datensätze?
4. Inwiefern beeinflussen Variationen in der Prompt-Formulierung die Bewertungsergebnisse von *Second-Level Judging*?
5. Welche Auswirkungen haben Second-Level Judging und Dynamic Prompting auf die Effizienz des Bewertungssystems hinsichtlich Rechenaufwand, Laufzeit und Kosten?

4 Vorgehensweise

Im Rahmen dieser Arbeit wird zunächst eine Literaturrecherche zu bestehenden Ansätzen im Bereich LLM-as-a-Judge durchgeführt. Dabei werden insbesondere Single-Prompt Baselines, Second-Level Judging und Dynamic Prompting betrachtet. Ziel ist es, den aktuellen Stand der Forschung zu analysieren, die Stärken und Schwächen der verschiedenen Ansätze zu identifizieren sowie geeignete Evaluationsmethoden und Datensätze auszuwählen.

Die Literatur zeigt, dass LLM-basierte Bewertungssysteme zwar eine skalierbare Alternative zu menschlichen Bewertungen darstellen [5], ihre Zuverlässigkeit, Konsistenz und Stabilität jedoch weiterhin Herausforderungen darstellen [1], [3], [6]. Darüber hinaus fehlen bislang systematische Vergleiche von Second-Level Judging und Dynamic Prompting unter einheitlichen Evaluationsbedingungen, insbesondere im Kontext sicherheitsrelevanter Aufgaben. Ausgehend von dieser Forschungslücke werden geeignete Benchmarks ausgewählt, um die verschiedenen Bewertungsansätze unter vergleichbaren Bedingungen zu untersuchen.

Der Fokus liegt dabei auf sicherheitsrelevanten Klassifikationsaufgaben. Hierzu werden Datensätze verwendet, die beispielsweise die Unterscheidung zwischen sicheren und unsicheren Antworten oder zwischen korrekten und inkorrekten Inhalten ermöglichen. Zusätzlich werden Datensätze berücksichtigt, die Antworten enthalten, welche durch Prompt-Injection- oder Jailbreak-Eingaben beeinflusst wurden.

Auf Basis dieser Datensätze wird ein experimentelles Vergleichsdesign umgesetzt, in dem Single-Prompt Baselines, Second-Level Judging und Dynamic Prompting miteinander verglichen werden. Alle Ansätze werden unter identischen Bedingungen auf denselben Daten getestet. Darüber hinaus werden verschiedene Prompt-Varianten sowie mehrere Durchläufe eingesetzt, um die Stabilität und Konsistenz der Bewertungsergebnisse zu analysieren.

Für die Experimente werden mehrere Sprachmodelle verwendet, darunter frei verfügbare Modelle sowie eine leistungsstärkere Referenz. Dadurch kann untersucht werden, welchen Einfluss die Wahl des Modells auf die Bewertungsergebnisse hat. Die Bewertung erfolgt anhand verschiedener Metriken. Zur Messung der Bewertungsqualität werden Accuracy und F1-Score herangezogen. Die Stabilität wird anhand der Variabilität der Ergebnisse über mehrere Durchläufe untersucht. Zusätzlich werden Effizienzmetriken wie Laufzeit, Anzahl der Modellaufrufe und die daraus entstehenden Kosten betrachtet. Um die Reproduzierbarkeit sicherzustellen, wird der gesamte Evaluationsprozess automatisiert und systematisch dokumentiert.

...

5 Planung

5.1 Eigene Vorkenntnisse

Im Rahmen des Studiums wurden grundlegende Kenntnisse in der Programmierung mit Python erworben, die insbesondere für die Umsetzung der experimentellen Auswertung genutzt werden. Darüber hinaus bestehen erste Erfahrungen im Bereich Machine Learning, die im Rahmen von Lehrveranstaltungen und Praktika, insbesondere im Bereich Deep Learning, gesammelt wurden.

Diese Vorkenntnisse umfassen grundlegende Konzepte wie das Training und die Evaluation von Modellen sowie den Umgang mit Datensätzen. Zudem wurden erste Erfahrungen im Umgang mit Schnittstellen (APIs) im Rahmen von Programmieraufgaben, unter anderem in Java, gesammelt.

Für die vorliegende Arbeit sind insbesondere die Fähigkeiten zur Datenverarbeitung, zur Implementierung von Experimenten sowie zur strukturierten Auswertung von Ergebnissen relevant. Diese werden im Rahmen der Arbeit weiter angewendet und vertieft.

Zusätzlich werden im Verlauf der Arbeit neue Kenntnisse erworben, insbesondere im Bereich der Bewertung von Large Language Models, fortgeschrittener Prompting-Strategien (z. B. Second-Level Judging und Dynamic Prompting) sowie der systematischen Durchführung und Analyse von Benchmark-Experimenten. Auch Aspekte wie die Automatisierung von Experimenten und die Analyse von Kosten und Effizienz werden dabei eine wichtige Rolle spielen.

...

5.2 Benötigte Ressourcen

- **Laptop:** Durchführung der Experimente sowie Datenverarbeitung und Auswertung.
- **Python und Bibliotheken:** Implementierung der Experimente, Verarbeitung der Daten und Analyse der Ergebnisse.
- **Zugriff auf Large Language Models:** Geplant ist die Nutzung von GPT-4o als Referenzmodell sowie ausgewählter Open-Source-Modelle wie Qwen und Llama. Diese Modelle zählen zu den am häufigsten verwendeten Judge-Modellen in aktuellen Arbeiten zu LLM-as-a-Judge [1]

und ermöglichen einen Vergleich zwischen kommerziellen und frei verfügbaren Modellfamilien. Sollte der Einsatz kostenpflichtiger APIs aufgrund von Ressourcen- oder Budgetbeschränkungen nur eingeschränkt möglich sein, können einzelne Experimente durch leistungsfähige Open-Source-Modelle ergänzt oder ersetzt werden.

- **Öffentliche Datensätze:** Grundlage für die Durchführung der Benchmark-Experimente.
- **Eigener Benchmark:** Ergänzende Datensätze zur Untersuchung spezifischer Fragestellungen.
- **Budget für API-Nutzung:** Abdeckung der Kosten für kostenpflichtige Modellzugriffe. Die zu erwartenden Kosten hängen maßgeblich von den verwendeten Datensätzen, der Anzahl der Evaluationsdurchläufe sowie dem Umfang der Judge-Prompts ab. Insbesondere sicherheitsrelevante Datensätze mit Prompt-Injection-, Jailbreak- oder anderen adversarialen Szenarien können aufgrund längerer Eingaben und komplexerer Bewertungsprozesse einen deutlich höheren Tokenverbrauch verursachen. Eine erste Abschätzung wurde auf Basis der Datensätze TruthfulQA und BeaverTails durchgeführt. Unter der Annahme von etwa 100 Beispielen pro Datensatz, drei Bewertungsansätzen (Baseline, Second-Level Judging und Dynamic Prompting) sowie mehreren Wiederholungen zur Untersuchung der Stabilität ergibt sich ein erwartetes Volumen von etwa 1 bis 1.5 Millionen Input-Token (Tabelle 1). Die daraus resultierenden API-Kosten hängen vom jeweils verwendeten Modell ab. Eine Übersicht der geschätzten Kosten für die betrachteten Judge-Modelle ist in Tabelle 2 dargestellt.

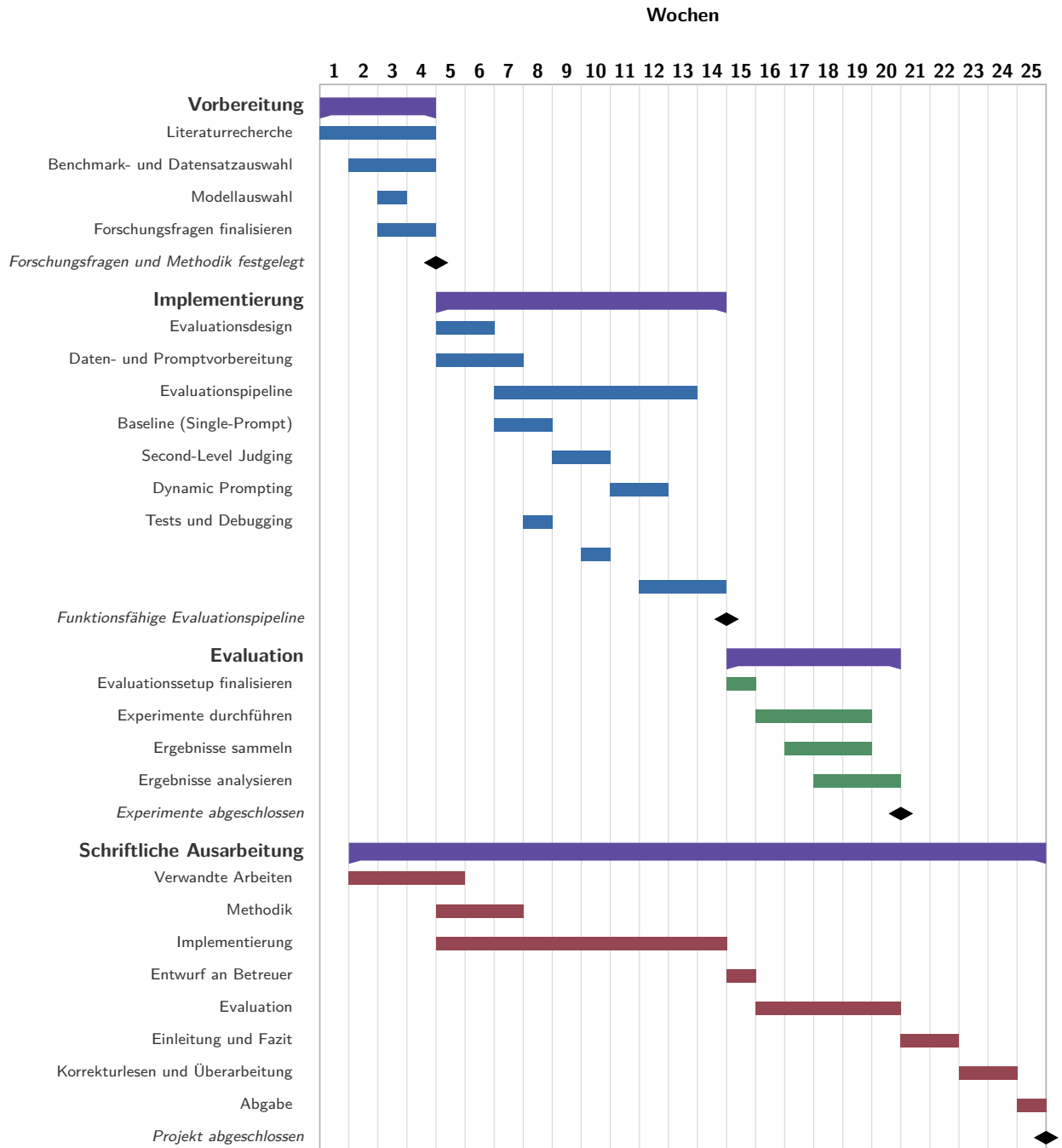
Tabelle 1: Abschätzung des Tokenverbrauchs für die Evaluation

Prompt-Typ	Input-Token	Gesamt Input	Output-Token	Gesamt Output
Baseline	500	300 000	50	30 000
Second-Level	700	420 000	100	60 000
Dynamic	1000	600 000	100	60 000
Summe	–	1.32 M	–	0.15 M

Tabelle 2: Geschätzte API-Kosten der betrachteten Judge-Modelle

Modell	Input-Kosten	Output-Kosten	Gesamtkosten
GPT-4o	\$3.30	\$1.50	\$4.80
Claude Sonnet 4	\$3.96	\$2.25	\$6.21
Gemini 2.5 Flash	\$1.98	\$1.35	\$3.33

5.3 Zeitplanung



5.4 Risiken und Ausweichplan

- **Nicht verfügbare oder eingeschränkt nutzbare Ressourcen (LLM-APIs)**
Risiko: Budgetüberschreitungen, Rate Limits oder eingeschränkter Zugriff auf Modelle.
Ausweichplan: Nutzung kostengünstiger oder frei verfügbarer Modelle, Reduktion der Stichprobengröße sowie Caching von Ergebnissen.
- **Instabilität und geringe Reproduzierbarkeit von LLM-Ergebnissen**
Risiko: Hohe Varianz durch nicht-deterministisches Verhalten der Modelle.
Ausweichplan: Durchführung mehrerer Durchläufe, Fixierung von Parametern (z. B. Temperature) sowie Einsatz geeigneter Stabilitätsmetriken.
- **Keine signifikanten Verbesserungen durch mehrstufige Ansätze**
Risiko: Second-Level Judging und Dynamic Prompting liefern keine besseren Ergebnisse als die Baseline.
Ausweichplan: Analyse der Einflussfaktoren, Erweiterung der Evaluation um zusätzliche Metriken (z. B. Effizienz und Kosten) sowie qualitative Fallanalysen.
- **Verschlechterung der Ergebnisse durch Second-Level Judging**
Risiko: Zusätzliche Bewertungsschritte können zu inkonsistenten oder schlechteren Resultaten führen.
Ausweichplan: Anpassung der Prompt-Strategien, Vergleich mit einfacheren Methoden sowie Analyse der Ursachen für Verschlechterungen.
- **Hoher Implementierungsaufwand der Experiment-Pipeline**
Risiko: Verzögerungen durch technische Komplexität.
Ausweichplan: Iterative Entwicklung, modularer Aufbau sowie Nutzung bestehender Frameworks.
- **Ungeeignete oder nicht ausreichende Datensätze**
Risiko: Benchmarks decken die Forschungsfragen nicht ausreichend ab.
Ausweichplan: Kombination mehrerer Datensätze sowie Ergänzung durch einen eigenen Benchmark.
- **Kein klar überlegener Prompting-Ansatz identifizierbar**
Risiko: Es lässt sich kein eindeutig bester Ansatz bestimmen.
Ausweichplan: Analyse von Trade-offs (Qualität, Kosten, Stabilität) sowie Ableitung praxisnaher Empfehlungen.

Literatur

- [1] J. Gu, X. Jiang, Z. Shi u. a., “A Survey on LLM-as-a-Judge,” *The Innovation*, S. 101253, Jan. 2026. DOI: 10.1016/j.xinn.2025.101253
- [2] S. Han, G. Titericz Junior, T. Balough und W. Zhou, “Judges Verdict: A Comprehensive Analysis of LLM Judge Capability Through Human Agreement,” 2026. arXiv: 2510.09738 [cs.CL].
- [3] T. Biskupski und S. Kleber, “Evaluating the Reliability and Fidelity of Automated Judgment Systems of Large Language Models,” 2026. arXiv: 2603.22214 [cs.CR].
- [4] H. Silva, M. Mendes und H. G. Oliveira, “Meta-Judging with Large Language Models: Concepts, Methods, and Challenges,” 2026. arXiv: 2601.17312 [cs.CL].
- [5] L. Zheng u. a., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” in *Advances in Neural Information Processing Systems*, Bd. 36, 2023, S. 46595–46623.
- [6] A. S. Thakur, K. Choudhary, V. S. Ramayapally, S. Vaidyanathan und D. Hupkes, “Judging the Judges: Evaluating Alignment and Vulnerabilities in LLMs-as-Judges,” in *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM²)*, Association for Computational Linguistics, 2025, S. 404–430. Adresse: <https://aclanthology.org/2025.gem-1.33/>
- [7] M. Russinovich, A. Salem und R. Eldan, “Great, Now Write an Article About That: The Crescendo Multi-Turn LLM Jailbreak Attack,” in *34th USENIX Security Symposium (USENIX Security 25)*, Aug. 2025, S. 2421–2440.
- [8] L. Huynh u. a., “Detecting Code Vulnerabilities using LLMs,” in *2025 55th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2025, S. 401–414. DOI: 10.1109/DSN64029.2025.00047
- [9] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu und C. Zhu, “G-Eval: NLG Evaluation using Gpt-4 with Better Human Alignment,” in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, Dez. 2023, S. 2511–2522. DOI: 10.18653/v1/2023.emnlp-main.153
- [10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan und S. Yao, “Reflexion: language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, Bd. 36, 2023, S. 8634–8652.
- [11] A. Madaan u. a., “Self-Refine: Iterative Refinement with Self-Feedback,” 2023. arXiv: 2303.17651 [cs.CL].